

This is example of audio signal processing and visualization using Python.

✓ 1. Importing Libraries

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
from scipy.io.wavfile import read
```

- matplotlib.pyplot → for plotting graphs.
 - numpy → for numerical operations, like creating the time vector.
 - scipy.io.wavfile.read → to read a .wav audio file.
-

✓ 2. Reading the .wav File

```
samplerate, data = read(r'C:\Snehal\CDAC\August-24\PG-DBDA\python\Demos\Day12_Lib\Ring10.wav')
```

- read() reads the audio file.
 - It returns:
 - samplerate → number of samples per second (e.g., 44100 Hz)
 - data → the audio signal as a NumPy array (amplitude values)
-

✓ 3. Print the Audio Info

```
print(samplerate)
```

```
print(data)
```

- This prints the sampling rate (e.g., 44100 samples/sec)
 - And the audio data (a long array of numbers representing sound amplitude over time)
-

✓ 4. Calculate Duration

```
duration = len(data) / samplerate
```

- **len(data)** = total number of samples
- Divide by samplerate to get **duration in seconds**

✅ 5. Create a Time Vector

```
time = np.arange(0, duration, 1/samplerate)
```

- This generates a time axis from 0 to duration with steps of 1/samplerate
- This matches each data point to a timestamp (for plotting)

✅ 6. Plot the Audio Signal

```
plt.plot(time, data)
```

```
plt.xlabel('Time [s]')
```

```
plt.ylabel('Amplitude')
```

```
plt.title('Ring10.wav')
```

```
plt.show()
```

- Plots **amplitude vs time** — this is the **waveform**
- Adds axis labels and a title

Result:

You'll see a **waveform** of the Ring10.wav file, showing how sound amplitude changes over time. This is a basic form of **sound visualization**.

Let me know if you'd like to:

- Zoom into a part of the waveform
- Play the sound in Python
- Convert stereo to mono
- Or export data to Excel or CSV!